Chat2DB: Chatting to the Database with Interactive Agent Assisted Language Models

Boyan Xu[1†], Yuyuan Cai[1†], Shaobin Shi[1], Zhifeng Hao[1,2], Ruichu Cai[1*]
[1]*Guangdong University of Technology, China*, [2]*Shantou University, China*
hpakyim@gmail.com, taroballscai@hotmail.com, d7inshi@outlook.com
haozhifeng@stu.edu.cn, cairuichu@gmail.com

2025 IEEE 41st International Conference on Data Engineering (ICDE) | 979-8-3315-3603-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICDE65448.2025.00367

***Abstract*—Cross-domain Text-to-SQL necessitates the capability of semantic parsers to generalize to unseen databases, thus simplifying the process of creating natural language interfaces for databases. The existing Text-to-SQL parser exhibits limitations in its adaptability to new databases, and its execution accuracy is not sufficient for building conversational applications, typically necessitating further fine-tuning for specific databases. In this paper, we introduce Chat2DB, a conversational system designed for database interactions that enhances parser capabilities, rendering them applicable in real-world contexts. Within Chat2DB, we implement an interactive schema-ranking agent that optimizes the performance of LMs-based parsers cost-effectively. We further propose an adaptive retraining stage to allow trained Text-to-SQL parsers to quickly adapt to the target database. Experimental evaluations were conducted to validate the performance of the key components of Chat2DB. In the demonstration, we showcase the interactive visualization interface of Chat2DB to achieve more accurate querying of databases by natural language.**

***Index Terms*—Text-to-SQL, Language Models, Agent**

I. Introduction

Relational databases are repositories of significant volumes of valuable data. However, the retrieval of such data mandates substantial investment in time from users to proficiently master the Structured Query Language (SQL). The deployment of Text-to-SQL technology, which entails the automated conversion of natural language queries into SQL statements, for the creation of a Natural Language Interface for Databases (NLIDB) [1], [2], can significantly enhance data utilization. Text-to-SQL parsers developed for specific databases necessitate significant annotation costs and training costs but lack the reusability for application to unseen databases.

Recently, Cross-domain Text-to-SQL, which focuses on the generalization of the trained parser across unseen database schemas, has been identified as a more promising solution. However, the leading offline approaches presented on the leaderboard fail to surpass an execution accuracy of 80%, which cannot satisfy practical database application contexts. The performance bottleneck is mainly attributed to the following two limitations about domain generalization: The primary limitation in enhancing the generalizability of parsers lies in the challenge of effectively linking correspondences between natural language questions and database schemas, a problem well-known as question-schema linking. Incorrect linking usually accumulates errors resulting in incorrect query statements being generated. A secondary limitation is that the training process for the Text-to-SQL parser encompasses a variety of databases, a number of which exhibit substantial deviation from the domain of the targeted database.

Aiming to alleviate these above limitations, we design and implement **Chat2DB**, a conversational system for database interactions. Chat2DB is featured with two key processes, including *Interactive Question-Schema Linking* and *Adaptive Retraining*, aimed at augmenting the performance of language models(LMs) based Text-to-SQL parsers. In the *Interactive Question-Schema Linking* stage, we devise a schema-ranking agent that incorporates user engagement to enhance the LMs' ability to link question terms and database items. In the *Adaptive Retraining* stage, we can adapt the Text-to-SQL parser by retraining on the domain-related databases. In response to the prerequisites for building real-world database applications, we further devise a rejection module and a unified datasource module to enhance the generality and security of Chat2DB.

Contributions. The contributions of this work are as follows.

- The *Interactive Question-Schema Linking* stage with schema-ranking agent to pluggably boost question-schema linking for LMs-based Text-to-SQL parsers.
- The *Adaptive Retraining* stage can adapt the model to the target database by strengthening the samples from the relevant database.
- We demonstrate how Chat2DB employs two stages to enhance natural language database querying accuracy.

II. Chat2DB Overview

A. System Architecture

Chat2DB consists of three major components, including parser management module, rejection module, and unified datasource module, as shown in Figure 1. Parser management module is the core of Chat2DB, which integrates existing leading PLM-based and LLM-based text-to-SQL parsers, and enhances their performance under the target database through two key stages: (i) In the *Interactive Question-Schema Linking* stage, a schema-ranking agent first pre-sorts the database items then finalizes the selection through user collaboration; (ii) In the *Adaptive Retraining* stage, the checkpoint of the Text-to-SQL parser undergoes supervised fine-tuning with samples from more relevant databases. The rejection module filters inappropriate content and unrelated database queries. The

†Equal Contribution, *Corresponding author

2375-026X/25/$31.00 ©2025 IEEE
DOI 10.1109/ICDE65448.2025.00367

Authorized licensed use limited to: Zhejiang University. Downloaded on November 03,2026 at 08:00:22 UTC from IEEE Xplore. Restrictions apply.

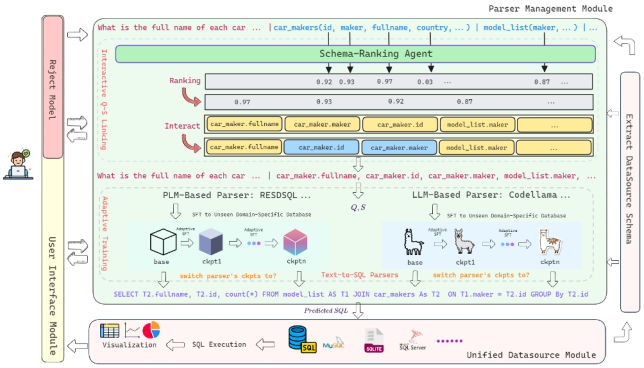


Fig. 1. Chat2DB Architecture Overview

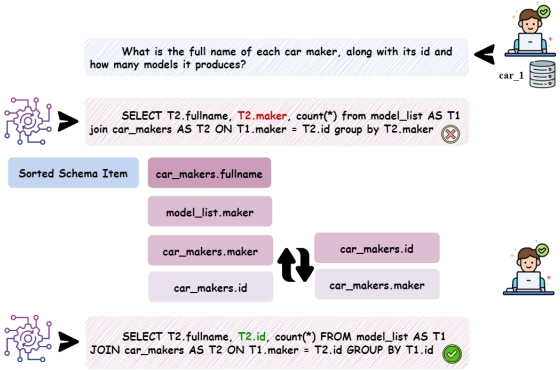


Fig. 2. Interactive Qustion-Schema Linking

unified datasource module manages different datasource like SQLite3, MySQL and SQL Server as the underlying database engine to build a unified data access endpoints for the system.

B. Parser Management Module

Text-to-SQL Parser. Given a natural language question $\mathcal{Q} = \{q_i\}_{i=1}^{|\mathcal{Q}|}$ and a database $\mathcal{D}$ with the corresponding database schemas $\mathcal{S} = \langle T, C \rangle$ including multiple tables $T = \{t_i\}_{i=1}^{|T|}$ and columns $C = \{c_1^{t1}, c_2^{t1}, ..., c_1^{t2}, c_2^{t2}, ...\}$. To flatten the given question $\mathcal{Q}$ and schema $\mathcal{S}$ into serialized inputs $x = \mathcal{Q}|\mathcal{S}$, the goal is to predict the SQL query y. The leading parsers are mainly divided into two types: PLM-based parsers and LLM-based parsers, the latter of which can be further divided into open-source LLM parsers and closed-source GPT series parsers. However, using GPT-based parsers for NLIDBs risks data leakage, so Chat2DB opts for the parsers built on open-source models.

$$h = \mathbf{Enc}(x); y = \mathbf{Dec}(h) \quad (1)$$

$$y = \mathbf{Dec}(x) \quad (2)$$

Formula 1 illustrates the training mechanism for PLM-based parsers within an encoder-decoder architecture, the decoder uses the encoded hidden state h to predict query y. Formula 2 delineates the training mechanism for LLM-based parsers within a decoder-only architecture, the decoder gets the full query y by constantly predicting the next token. It can be seen that both parsers jointly encode the database items in S with query words in Q to capture the linking relationships between them. Therefore, reducing the impact of irrelevant items before input is a simple and effective approach.

Interactive Question-Schema Linking. Inspired by RESDSQL [3], an additional model was used to optimize the question-schema linking process by specifically ranking the database items. Our key observation is that the ranking model effectively filters irrelevant items but retains items in an inaccurate order. Based on the above observations, we designed an interactive schema ranking agent which can assist the user to regenerate predicted SQL as shown in Figure 2. Specifically, we firstly put x into **RoBERTa** [4] and aggregated it using two layers of **BiLSTM** and nonlinear layer. Each table embedding that can be obtained after pooling can be represented as $T_i \in \mathbb{R}^{1\times d}$ $(i \in \{1, ..., N\})$, and each column embedding can be represented as $C_k^i \in \mathbb{R}^{1\times d}$ $(i \in \{1, ..., N\}, k \in \{1, ..., n_i\})$. Then we enhance the representation of the table through a column-enhanced Layer. The principle is to use multi-head scaled dot-product attention layer to inject the information of the Column into the embedding of the corresponding table. It can be represented as $\hat{T}_i = Norm(T_i + MultiHeadAttn(T_i, C_:^i, C_:^i, h))$, where $C_:^i \in \mathbb{R}^{1\times d}$ and h is the number of heads. Subsequently, by inputting $\hat{T}_i$ and C_k^i into two layers of **MLP** and Softmax, we can derive the probability values for each table along with its respective column names. Chat2DB will arrange the schema by sorting from largest to smallest, focusing on the top-4 tables and their corresponding top-5 columns. Chat2DB will then display the sorted outcome list to the endpoint user for feedback. User can choose whether to adjust the order of schema items based on this result. Tables and columns selected by the user will have a probability value of 1 assigned to them, get rearranged, and return back to the parser.

Adaptive Retraining. Moreover, due to the difference between the training corpus and the unseen domain-specific database, the aforementioned interactive question-schema linking alone is insufficient for generalization. Fine-tuning on a corpus similar to the current database schema would significantly enhance parser performance. Therefore, as shown in figure 3, we propose an *Adaptive Retraining* stage, which first finds the closest training corpus, and then performs further adaptive retraining based on the weights of the existing parser model. Specifically, for the schema list S_A of each training corpus and the schema list S_C of the current database, we first calculate the embedding distance of each item between the two lists based on Sentence-Bert, and set a filtering threshold to filter out the most relevant training schema $S_{A'}$, where $S_{A'}$ is also the subset of S_A. We further refine the training corpus τ based on $S_{A'}$, resulting in a more detailed training corpus, denoted as τ'. Once Chat2DB collects enough detailed training corpus τ', Chat2DB will automatically perform adaptive retraining according to the existing model weights and the

Authorized licensed use limited to: Zhejiang University. Downloaded on November 03,2026 at 08:00:22 UTC from IEEE Xplore. Restrictions apply.

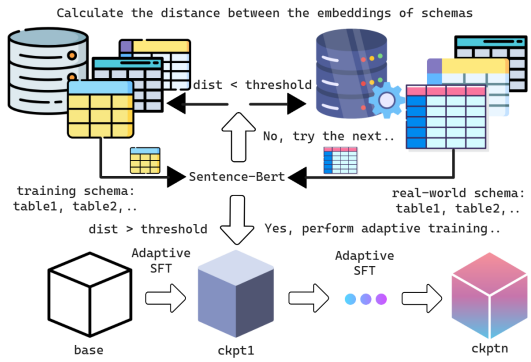


Fig. 3. Adaptive Retraining Stage

most relevant training corpus τ'. Otherwise, Chat2DB will not perform the retraining process due to the lack of a sufficiently detailed training corpus. When training is completed, the user can selectively switch specific checkpoints to achieve better generalization.

C. Rejection Module

We have designed an additional filter process to reduce and reject invalid inferences. Specifically, we first use a content review service to filter inappropriate content, then calculate the cosine similarity of text embeddings with Sentence-BERT and reject inputs with a similarity score below a predefined threshold δ. Formally, we define the similarity score as:

$$\text{score} = \frac{E_{\mathcal{Q}} \cdot E_{\mathcal{S}}}{||E_{\mathcal{Q}}||_2 \cdot ||E_{\mathcal{S}}||_2}$$

where $E_{\mathcal{Q}}$ and $E_{\mathcal{S}}$ represent the embeddings of the question and schema, respectively, and $||\cdot||_2$ denotes the Euclidean norm. Additionally, following [5], we have adopted a series of security rules to further enhance the overall security.

D. Unified Datasource Module

Unified Datasource Module is a critical component designed to consolidate and manage a variety of existing data sources, including SQLite3, MySQL, SQL Server, among others, into a unified data access point which allows for a streamlined process of data access and manipulation, reducing the complexity and time required to manage multiple databases separately. As the fundamental underlying database engine, it enables Chat2DB the capability to extract all metadata schema information from a range of databases in a uniform manner and execute predicted SQL to deliver results.

E. Experimental Evaluation

We take RESDSQL [3] as the representative of the PLM-based parser and Codellama [6] as the representative of the LLM-based parser. We conduct all our experiments on the **Spider** dataset. Spider is a cross-domain text-to-SQL dataset. We classify SQL queries into three levels: **EASY** for single-table, **MEDIUM** for multi-table, and **HARD** for those with keywords like ORDER BY, GROUP BY, etc

TABLE I
THE EXECUTION ACCURACY ON THE SPIDER DEV DATASET.

PARSER	EASY(%)	MEDIUM(%)	HARD(%)	EX(%)
PLM-Based	91.9	81.8	74.7	79.4
+ classified	93.1	85.2	75.3	82.1
+ interactive	96.4	88.6	77.0	85.4
LLM-Based	85.5	73.3	67.8	72.7
+ classified	95.2	89.9	77.6	84.3
+ interactive	96.0	91.7	83.3	87.4

TABLE II
THE EXECUTION ACCURACY ON THE SPIDER TEST DATASET.

PARSER	EASY(%)	MEDIUM(%)	HARD(%)	EX(%)
PLM-Based	93.8	86.1	73.6	81.7
+ additive	93.8	88.9 (+2.8)	69.8	82.2(+0.5)
LLM-Based	93.8	83.3	69.8	82.7
+ additive	93.8	84.7 (+1.4)	77.6(+7.8)	83.2(+0.5)

Experiment 1: Comparison of Normal Linking and Interactive Linking. To illustrate the effectiveness of normal schema linking versus interactive schema linking, we conduct an experiment, that examines the generalization performance of PLM-based and LLM-based parsers in completing Text-to-SQL under three conditions: without any linking, with normal schema linking, and with interactive schema linking. As illustrated in Table I, the performance of the original model falls short of expectations, and the "classified" row indicates an improved outcome achieved by using a classifier, which surpasses the original model's results. Meanwhile, the "interactive" row represents the use of the Chat2DB's interactive schema-ranking agent, which delivers the best performance.

Experiment 2: Impact of Adaptive Retraining. To further elucidate the impact of the retraining, we randomly sample 3 databases from the Spider test set in order to better simulate cross-domain databases, and perform the adaptive retraining procedure. As is shown in Table II, experiments demonstrate that adaptive retraining can effectively improve the model's generalization performance on these databases.

III. SYSTEM DEMONSTRATION

In this demonstration[1], we showcase two scenarios. Scenario 1 provides an overview of an interactive conversation with Chat2DB, which includes these three major interactive steps: querying a database using natural language questions, participating in the schema item sorting process, and executing predicted SQL to return results. Scenario 2 is an overview of the adaptive retraining procedure on domain-specific database.

A. Scenario 1: Interactive Chatting

Figure 4 shows the user interaction interface of Chat2DB, comprising a database information panel on the left and a

[1]Demonstration video can be found in : https://youtu.be/feye9XSRFYM

Authorized licensed use limited to: Zhejiang University. Downloaded on November 03,2026 at 08:00:22 UTC from IEEE Xplore. Restrictions apply.

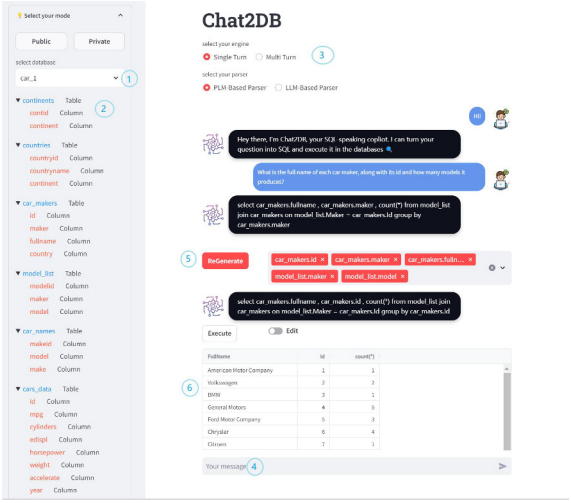


Fig. 4. Scenario1: Interactive Chatting With Database

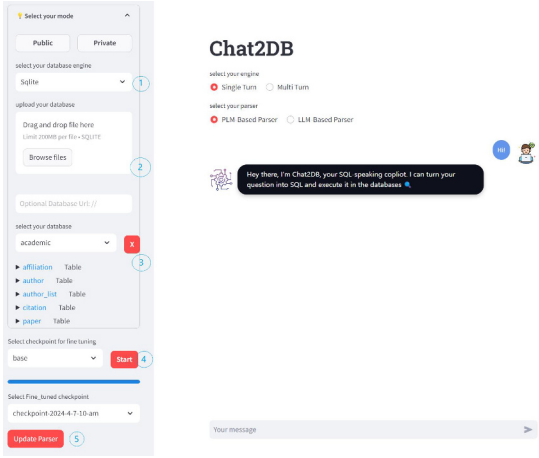


Fig. 5. Scenario2: Adaptive Retraining Stage

database interaction panel on the right. The information panel offers two modes: Public Databases and Domain-Specific Private Databases, allowing users to select a database that suits their needs. Users can expand the list by clicking ❶ and choose a database. ❷ Once selected, the relevant database schema and columns appear below as a helpful hint. For the database interaction panel, at the top of the panel are ❸ the rounds and parser options. Users can choose single or multi rounds based on the conversation, and select between PLM-Based Parser and LLM-Based Parser. When ❹ the user enters a natural language question, the system processes it and uses the parser to generate SQL. In addition, to further address the potential SQL inaccuracies, ❺ Chat2DB provides a list of schema items based on their relevance to the question, asking the user for feedback. Users can sort the list of candidates and check the "regenerate" button to feed it back to the Chat2DB. When system receives the feedback, it constructs a new question-schema linking input based on the user's feedback, thereby regenerating an accurate SQL statement. Finally, ❻ users can choose to execute predictive SQL and get the final results.

B. Scenario 2: Adaptive Retraining

Figure 5 shows the adaptive retraining panel for domain-specific databases. Currently, we support multiple datasources like SQLite3, MySQL, etc.., ❶ users can select the database engine from a drop-down list.. Then, ❷ users can choose to upload a database file or enter a database address to integrate a domain-specific database into Chat2DB. Meanwhile, to increase flexibility, ❸ users can customize which databases are integrated. After successfully connecting to Chat2DB, users can choose whether or not to fine-tune the current database to improve its generalization performance. The retraining process is automatically controlled by Chat2DB and ❹ user can also manually start it. Chat2DB will display the training process to the user interface. Finally, ❺ users can select the desired checkpoint as the primary parser once the training is complete.

IV. Conclusion

We propose Chat2DB, a conversational system designed for database interactions that enhances Text-to-SQL parser capabilities rendering them applicable in domain-specific contexts, through the well-designed *Interactive Question-Schema Linking* and *Adaptive Retraining*. In the demonstration, we illustrate the two-stage capabilities of Chat2DB with two scenarios, simulating the interactive process where a user chats with a database and demonstrating the adaptive retraining stage tailored to a domain-specific database.

Acknowledgment

This research was supported in part by National Science and Technology Major Project (2021ZD0111501), National Science Fund for Excellent Young Scholars (62122022), Natural Science Foundation of China (62406078, 62476163, U24A20233), Guangdong Basic and Applied Basic Research Foundation (2023B1515120020).

References

[1] J.-P. Zhu, P. Cai, B. Niu, Z. Ni, K. Xu, J. Huang, J. Wan, S. Ma, B. Wang, D. Zhang *et al.*, "Chat2query: A zero-shot automatic exploratory data analysis system with large language models," in *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 2024, pp. 5429–5432.

[2] S. Xue, D. Qi, C. Jiang, W. Shi, F. Cheng, K. Chen, H. Yang, Z. Zhang, J. He, H. Zhang *et al.*, "Demonstration of db-gpt: Next generation data interaction system empowered by large language models," *arXiv preprint arXiv:2404.10209*, 2024.

[3] H. Li, J. Zhang, C. Li, and H. Chen, "Resdsql: Decoupling schema linking and skeleton parsing for text-to-sql," *arXiv preprint arXiv:2302.05965*, Feb 2023.

[4] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized bert pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.

[5] X. Peng, Y. Zhang, J. Yang, and M. Stevenson, "On the security vulnerabilities of text-to-sql models," *arXiv preprint arXiv:2211.15363*, 2022.

[6] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin *et al.*, "Code llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.

Authorized licensed use limited to: Zhejiang University. Downloaded on November 03,2026 at 08:00:22 UTC from IEEE Xplore. Restrictions apply.